REPORT ON MAIN PROJECT

Name : Poovizhi R

Batch : TN_DA_FNB07

Contact No : +91 9865282680

Email ID : poovizhimathankumar@gmail.com

Project Title : Amazon E-Commerce Sales & Customer Analytics

Project Domain : Sales and E-commerce

Submission Date : 01-04-2026

Mentor Name : Kumaran M

Raw Dataset Link : https://drive.google.com/file/d/13yEbf5u7kX8oRJFGwXEh3ehooz0dm6ym/view?usp=drive_link

Cleaned Dataset Link : https://drive.google.com/file/d/1_mkHMc75BM1VnVH3i_hkuQob0zZPtPqT/view?usp=drive_link

Project Title

Amazon E-Commerce Sales & Customer Analytics: Forecasting, Market Trends, and Revenue Insights using Python(Pandas, NumPy, Matplotlib, Seaborn)

Domain

Sales and E-commerce

Objective

1. To clean, transform, and preprocess the dataset to ensure data quality and consistency for analysis

2. To perform Exploratory Data Analysis (EDA) to identify meaningful patterns, trends, and relationships in sales transactions

3. To create clear and meaningful visualizations that effectively communicate the story behind the data

4. To analyze customer purchasing behavior based on location, product category, brand, and payment method

5. To evaluate the impact of discounts and shipping costs on

total sales and overall profitability

6. To provide actionable insights and strategic recommendations to improve profit and sales performance.

Outcome

The project provides clear insights into Amazon sales data through data analysis and visualization.It identifies top-performing products, categories, and customer purchasing patterns.The results help in predicting future sales and improving business strategies.

Dataset Information

Source: Kaggle

Year / Timeline: Data collected during 2026

Dataset Description:

1. Order_id - Unique identification number assigned to each order.
2. Order_date - The date when the customer placed the order.
3. Ship_date - The date when the order was shipped from the warehouse.
4. Delivery_date - The date when the product was delivered to the customer.
5. Order_status - Current status of the order (Delivered, Shipped, Cancelled, Pending).
6. Customer_id - Unique identification number for each customer.
7. Customer_name - Name of the customer who placed the order.
8. Country - Country where the customer is located.
9. State - State or region of the customer.
10. City - City where the customer lives.
11. Product_id - Unique identification number for each product.
12. Product_name - Name of the product purchased.
13. Category - Main category to which the product belongs (e.g., Electronics, Clothing,Fashion).
14. Sub_category - Sub-division of the main product category.
15. Brand - Brand or manufacturer of the product.
16. Quantity - Number of units of the product purchased in the order.
17. Unit_price - Price of a single unit of the product.
18. Discount - Discount applied to the product price.
19. Shipping_cost - Cost charged for delivering the product.
20. Total_sales - Total sales amount for the order after including quantity, discount, and shipping cost.
21. Payment_method - Method used by the customer to make payment (Card,COD, UPI,NetBanking).

Type of Analysis

- **Descriptive Analysis** - Descriptive analysis summarizes the dataset using basic statistics and visualizations.It helps understand overall sales performance, product categories, and customer distribution.This analysis provides a clear overview of the Amazon sales data.

- **Diagnostic Analysis** - Diagnostic analysis identifies the reasons behind sales patterns and trends.It examines factors such as discounts, shipping cost, product category, and location.This helps understand why certain products or regions generate higher sales.
- **Prescriptive Analysis** - Prescriptive analysis provides recommendations based on the analysis results.It suggests strategies such as optimizing discounts, improving product focus, and targeting customers.This helps businesses improve profitability and overall sales performance.

Stages for DA Project

Stage 1 – Problem Definition and Dataset Selection

Code:

```
from google.colab import drive

drive.mount('/content/drive')
```

Output:

```
Mounted at /content/drive
```

Code:

```
#Import libraries

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

Code:

```
#Load Dataset
df=pd.read_csv('/content/drive/MyDrive/Main Project/Raw dataset_Amazon_sales_dataset.csv')
```

code:

```
# Dataset description (rows, columns, features)
#Dataset shape (rows,columns)
df.shape
```

Output:

```
(10000, 21)
```

Code:

```
# Column names
df.columns
```

Output:

```
Index(['order_id', 'order_date', 'ship_date', 'delivery_date', 'order_status',
       'customer_id', 'customer_name', 'country', 'state', 'city',
       'product_id', 'product_name', 'category', 'sub_category', 'brand',
       'quantity', 'unit_price', 'discount', 'shipping_cost', 'total_sales',
       'payment_method'],
      dtype='object')
```

Code:

```
#Initial EDA (head, info, describe, shape, null checks, duplicate check)
#View First Five Rows
df.head()
```

Output:

```
5 rows × 21 columns
```

Code:

```
#Dataset Information
df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   order_id       10000 non-null  object
 1   order_date     10000 non-null  object
 2   ship_date      10000 non-null  object
 3   delivery_date  10000 non-null  object
 4   order_status   10000 non-null  object
 5   customer_id    10000 non-null  object
 6   customer_name  10000 non-null  object
 7   country        10000 non-null  object
 8   state          10000 non-null  object
 9   city           10000 non-null  object
 10  product_id     10000 non-null  object
 11  product_name   10000 non-null  object
 12  category       10000 non-null  object
 13  sub_category   10000 non-null  object
 14  brand          10000 non-null  object
 15  quantity       10000 non-null  int64
 16  unit_price     10000 non-null  float64
 17  discount       10000 non-null  float64
 18  shipping_cost  10000 non-null  float64
 19  total_sales    10000 non-null  float64
 20  payment_method 10000 non-null  object
dtypes: float64(4), int64(1), object(16)
memory usage: 1.6+ MB
```

Code:

```
#Statics summary
df.describe()
```

Output:

	quantity	unit_price	discount	shipping_cost	total_sales
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000
mean	3.01440	25126.511133	0.149968	85.053192	64212.910555
std	1.42035	14343.922332	0.086828	37.575284	50992.635082
min	1.00000	214.200000	0.000000	20.010000	309.939600
25%	2.00000	12657.827500	0.070000	52.507500	24037.196775
50%	3.00000	24880.490000	0.150000	84.995000	50287.177500
75%	4.00000	37544.640000	0.220000	117.680000	93417.891825
max	5.00000	49981.880000	0.300000	149.950000	249155.530000

Code:

```
#missing values
df.isnull().sum()
```

Output:

	0
order_id	0
order_date	0
ship_date	0
delivery_date	0
order_status	0
customer_id	0
customer_name	0
country	0
state	0
city	0
product_id	0
product_name	0
category	0
sub_category	0
brand	0
quantity	0

	0
unit_price	0
discount	0
shipping_cost	0
total_sales	0
payment_method	0

dtype: int64

Code:

```
#Duplicate records
df.duplicated().sum()
```

Output:

np.int64(0)

Stage 2 – Data Cleaning and Pre-processing

Code:

```
#Missing values
df.isnull().sum()
```

Output:

	0
order_id	0
order_date	0
ship_date	0
delivery_date	0
order_status	0
customer_id	0
customer_name	0
country	0

	0
state	0
city	0
product_id	0
product_name	0
category	0
sub_category	0
brand	0
quantity	0
unit_price	0
discount	0
shipping_cost	0
total_sales	0
payment_method	0

dtype: int64

Code:

```
#Handling Duplicate Records
df.duplicated().sum()
```

Output:

```
np.int64(0)
```

Code:

```
#summary statistics
df.iloc[:,17:].describe()
```

Output:

0	discount	shipping_cost	total_sales
count	10000.000000	10000.000000	10000.000000
mean	0.149968	85.053192	64212.910555
std	0.086828	37.575284	50992.635082
min	0.000000	20.010000	309.939600

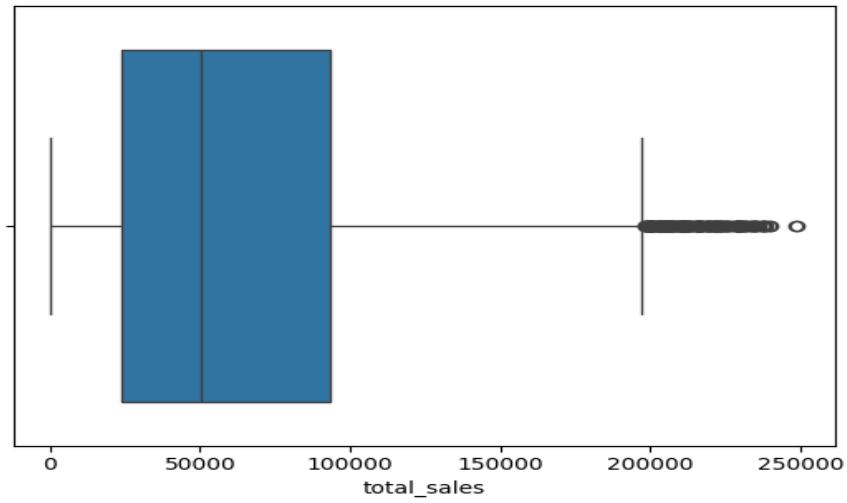
0	discount	shipping_cost	total_sales
25%	0.070000	52.507500	24037.196775
50%	0.150000	84.995000	50287.177500
75%	0.220000	117.680000	93417.891825
max	0.300000	149.950000	249155.530000

Code:

```
#Outliers

sns.boxplot(x=df['total_sales'])
plt.show()
```

Output:



Code:

```
#Calculate skewness of all numeric columns
df.skew(numeric_only=True)
```

Output:

	0
quantity	-0.015970
unit_price	0.002927

	0
discount	-0.004165
shipping_cost	0.003982
total_sales	0.963536

dtype: float64

Code:

```
#Log transformation
df['total_sales_log'] = np.log1p(df['total_sales'])

#Check skewness
df[['total_sales', 'total_sales_log']].skew()
```

Output:

	0
total_sales	0.963536
total_sales_log	-1.016215

dtype: float64

Code:

```
#convert data types
df['order_date'] = pd.to_datetime(df['order_date'])
df['ship_date'] = pd.to_datetime(df['ship_date'])
df['delivery_date'] = pd.to_datetime(df['delivery_date'])

#delivery days
df['delivery_days']=(df['delivery_date']-df['order_date']).dt.days
```

Code:

```
df[['order_date','delivery_date','delivery_days']]
```

Output:

	order_date	delivery_date	delivery_days
0	2026-01-31	2026-01-08	-23
1	2026-01-20	2026-02-03	14
2	2026-01-15	2026-01-03	-12
3	2026-01-18	2026-01-20	2
4	2026-01-27	2026-01-23	-4
...	...	...	...
9995	2026-02-08	2026-01-10	-29
9996	2026-01-17	2026-02-03	17
9997	2026-01-17	2026-01-15	-2
9998	2026-01-08	2026-02-07	30
9999	2026-01-31	2026-02-08	8

10000 rows × 3 columns

Code:

```
#derived features-revenue after discount
df['final_price']=df['unit_price']-df['discount']

#view specific columns
df[['unit_price','discount','final_price']].head()
```

Output:

	unit_price	discount	final_price
0	42467.79	0.26	42467.53
1	36138.76	0.24	36138.52
2	47148.93	0.14	47148.79
3	18487.99	0.06	18487.93
4	1742.25	0.10	1742.15

Code:

```
#total revenue
df['calculated_sales']=df['quantity']*df['final_price']
```

```
#New features data converstion for unit_price,discount,total_sales
df['total_sales_round']=df['total_sales'].round(2)
df['unit_price_round']=df['unit_price'].round(2)
df['discount_percent']=(df['discount']*100).round(0)
```

Code:

```
df.head()
```

Output:

5 rows × 28 columns

Code:

```
#column names

df.columns
```

Output:

```
Index(['order_id', 'order_date', 'ship_date', 'delivery_date', 'order_status',
       'customer_id', 'customer_name', 'country', 'state', 'city',
       'product_id', 'product_name', 'category', 'sub_category', 'brand',
       'quantity', 'unit_price', 'discount', 'shipping_cost', 'total_sales',
       'payment_method', 'total_sales_log', 'delivery_days', 'final_price',
       'calculated_sales', 'total_sales_round', 'unit_price_round',
       'discount_percent'],
      dtype='object')
```

Code:

```
df.to_csv('cleaned_amazon_dataset.csv',index=False)
```

Stage 3 – EDA and Visualizations

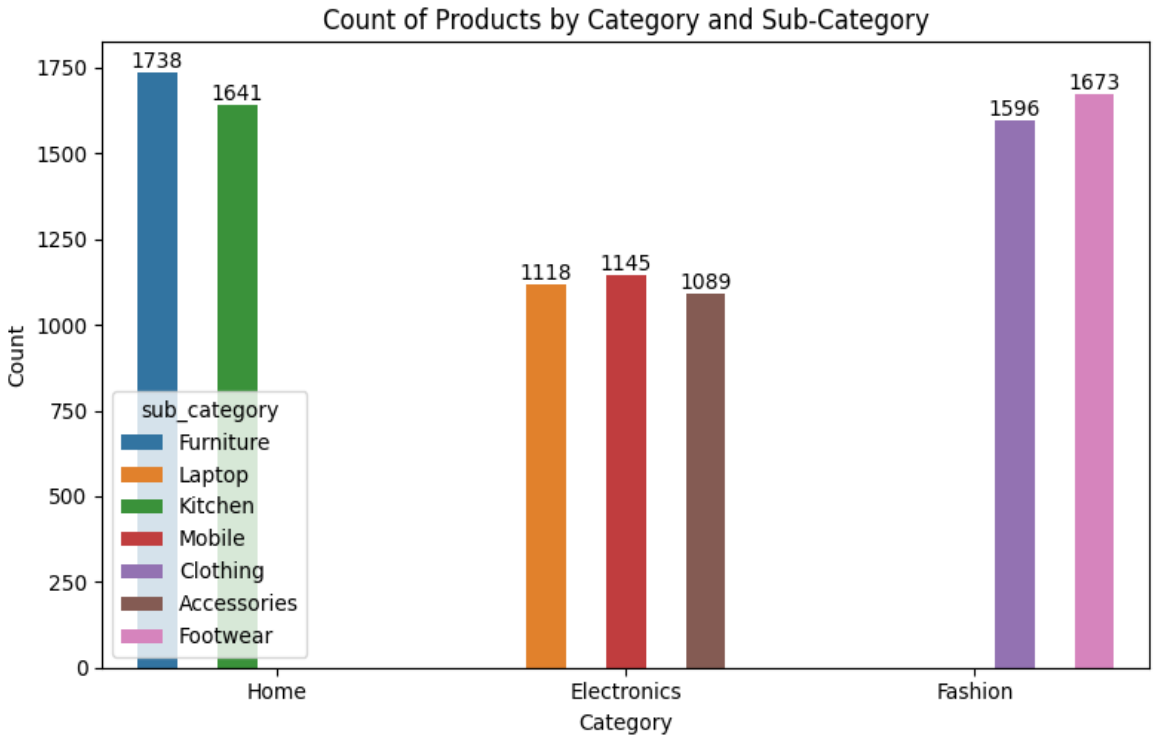
1.Univariate Analysis → distribution of single variables (countplot, histogram, boxplot)

Code:

```
# Univariate Analysis
# 1.Countplot

plt.figure(figsize=(8,5))
ax=sns.countplot(x='category',hue='sub_category',data=df)
for container in ax.containers:
    ax.bar_label(container)
plt.title('Count of Products by Category and Sub-Category')
plt.xlabel('Category')
plt.ylabel('Count')
plt.tight_layout()
#plt.grid(axis='y',linestyle='--',alpha=0.6)
plt.show()
```

Output:



Interpretation of your above chart

Explain the Chart:

- This is a countplot showing the distribution of products across different categories and sub-categories in amazon sales dataset.
- X-axis → Category (Home, Electronics, Fashion)
- Y-axis → Count of products
- Hue -> Sub-category(Furniture,Kitchen,Laptop,Mobile,etc.)

What is it saying?

- Home category

  -> Furniture: 1738

  -> Kitchen: 1641

  -> Total: 3379
- Electronics

  -> Laptop: 1118

  -> Mobile: 1145

  -> Accessories: 1089

  -> Total: 3352
- Fashion

  -> Clothing: 1596

  -> Footwear: 1673

  -> Total: 3269
- All three categories have almost equal distribution.No category is heavily dominant.

What features you used?

- category → Main categorical variable
- sub_category → Secondary categorical variable (via hue)
- Visualization → Seaborn countplot
- Additional: bar_label() used to show exact counts on bars

From this what you are showing us?

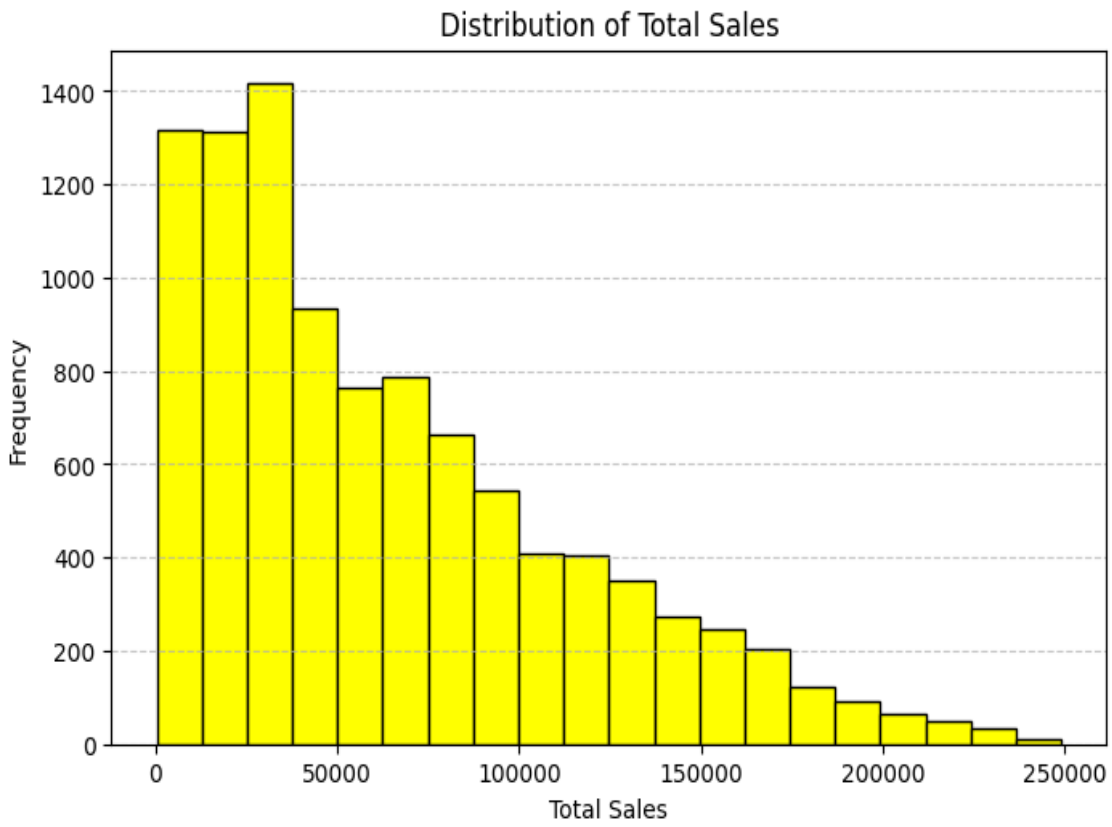
- The dataset is well-balanced across categories

- No category is heavily overrepresented -> good for fair analysis & modeling
- Each category has internal diversity (sub-categories)

2.Histogram

```
plt.figure(figsize=(8,5))
plt.hist(df['total_sales_round'],bins=20,edgecolor='black',color='yellow')
plt.xlabel('Total Sales')
plt.ylabel('Frequency')
plt.title('Distribution of Total Sales')
plt.xlabel('Total Sales')
plt.ylabel('Frequency')
plt.grid(axis='y',linestyle='--',alpha=0.7)
plt.tight_layout
plt.show()
```

Output:



Interpretation of your above chart

Explain the Chart:

- This is a Histogram showing the distribution of total_sales_round values.
- It shows the frequency distribution of total_sales_round.
- Data is divided into continuous intervals (bins).
- It helps to understand pattern,spread,and shape of data.
- Used for numerical data analysis

What is it saying?

- Data is not evenly distributed
- The data is right-skewed (positively skewed)
- Most sales values are in the lower range(0-80,000 approx.)
- Very high sales values are less frequent

What features you used?

- Column used: total_sales_round (numerical feature)
- X-axis: Total Sales values

- Y-axis: Frequency (count of records)
- Bins: 20 (grouping of values)
- Grid lines: Improve readability
- Edgecolor: Highlights bin separation
- Figure size: Better visualization

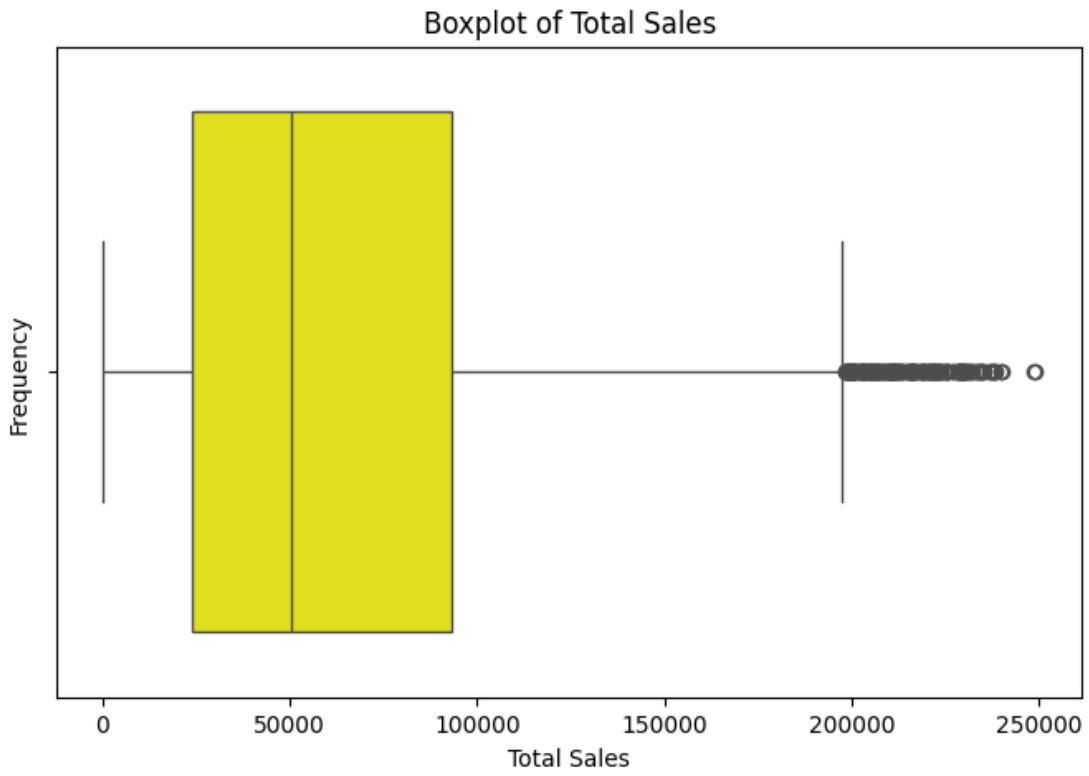
From this what you are showing us?

- Majority of orders are low to medium value
- Only few orders generate very high sales
- Sales distribution is not normal (skewed)
- Helps identify outliers and customer buying pattern
- Useful for data preprocessing
- Helps business to analyze sales behavior

#3.Boxplot

```
plt.figure(figsize=(8,5))
sns.boxplot(x=df['total_sales_round'],color='Yellow')
plt.title('Boxplot of Total Sales')
plt.xlabel('Total Sales')
plt.ylabel('Frequency')
plt.show()
```

Output:



Interpretation of your above chart

Explain the Chart:

- This is a Boxplot used to visualize the distribution and outliers of total_sales_round
- It shows median, quartiles, spread, and extreme values
- The box represents the middle 50% of the data (IQR- Interquartile Range)
- The line inside the box represents the median (central value)
- The whiskers show the spread of data excluding outliers
- Points outside whiskers represent outliers (extreme values)

What is it saying?

- The data is positively skewed (right-skewed)
- The median lies closer to the lower range
- The right whisker is longer → indicates higher values spread
- There are many outliers on the right side (high sales values)
- Most sales are within a moderate range, but some are very high

What features you used?

- Column used: total_sales_round (numerical feature)
- Median (middle line) → center of data

- Box (Q1 to Q3) → middle 50% of data
- Whiskers → range without outliers
- Outliers (dots) → extreme values

From this what you are showing us?

- Majority of sales fall in the lower to mid range
- There are many high-value outliers
- Sales distribution is not normal
- Helps identify extreme transactions
- Useful for Outlier treatment,Data transformation,Business insights.

```
# Bivariate Analysis
# 4. Scatterplot(Price vs Sales)

plt.figure(figsize=(8,5))
sns.scatterplot(x='unit_price_round',y='total_sales_round',hue='category',data
=df,color='green',alpha=0.7)
plt.xlabel('unit Price')
plt.ylabel('Total Sales')
plt.title('Unit Price vs Total Sales')
plt.grid(True,linestyle='--',alpha=0.5)
plt.show()
```

Output:

Interpretation of your above chart

Explain the Chart:

- This is a bivariate analysis chart using a scatterplot.
- The scatterplot shows the relationship between Unit Price (X-axis) and Total Sales (Y-axis).
- Each point represents a transaction, and different colors represent different product categories.

What is it saying?

- There is a strong positive relationship between Unit Price and Total Sales
- As the unit price increases, the total sales also increase
- The points form a triangular pattern,which indicates:
- Total sales depend on both price and quantity purchased.
- Different colors (categories) show that:

  -> Some categories perform better at higher price ranges

-> Some categories are concentrated in lower price ranges

What features you used?

- Scatterplot → to analyze relationship between two variables
- Hue = 'category' → to compare multiple product categories
- unit_price_round (Independent variable)
- total_sales_round (Dependent variable)
- Visualization features:

    * color='green' → for better visibility

    * alpha=0.7 → transparency to handle overlapping points

    * grid=True → improves readability

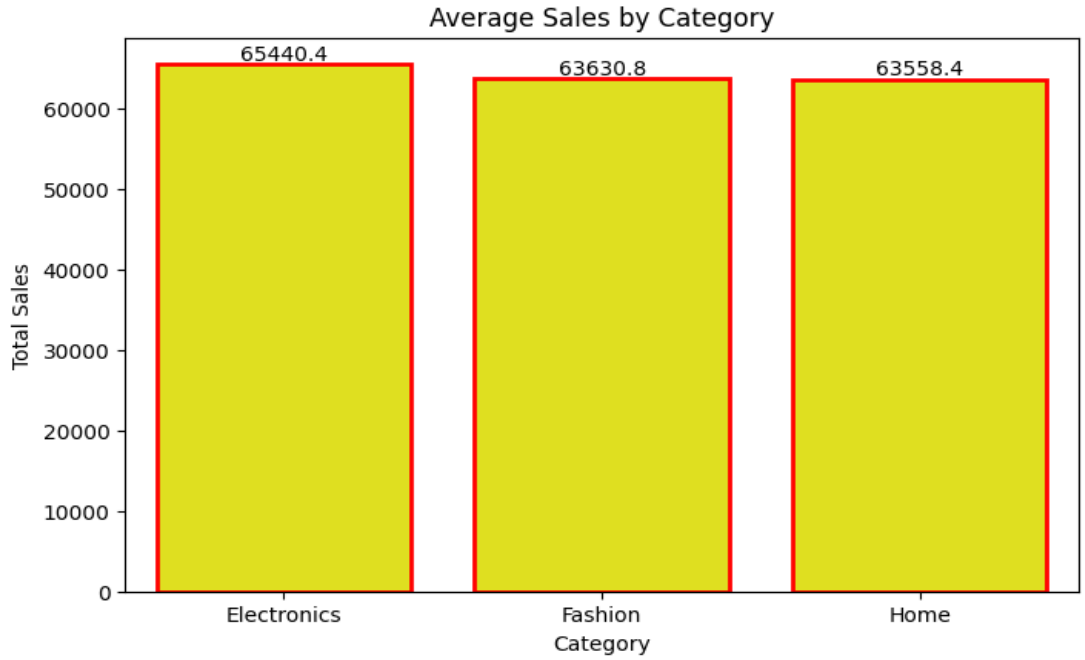
From this what you are showing us?

- Higher price → higher total sales value (positive relationship)
- But sales are not consistent at each price level
- This means sales also depend on Quantity sold,Customer demand,Discounts.
- Businesses must balance price and sales volume to maximize revenue.

5.Barplot (Category vs Sales)

```
plt.figure(figsize=(8,5))
grouped=df.groupby('category')['total_sales_round'].mean().reset_index()
bar=sns.barplot(x='category',y='total_sales_round',data=grouped,edgecolor='red',color='yellow',linewidth=2)
bar.bar_label(bar.containers[0])
plt.xlabel('Category')
plt.ylabel('Total Sales')
plt.title('Average Sales by Category')
plt.show()
```

Output:



Interpretation of your above chart

Explain the Chart:

This is a barplot showing average (mean) total sales by category.

- Categories: Electronics, Fashion, Home
- Y-axis shows the average sales value for each category.

What is it saying?

- Electronics has the highest average sales (65,440)
- Fashion and Home are slightly lower but very close
- Overall all categories perform almost equally.Difference is small not very significant.

What features you used?

- groupby().mean() → to calculate average sales per category
- sns.barplot() → to visualize category-wise comparison
- color='yellow', edgecolor='red' → for styling

- ax.bar_label() → to display values on bars

From this what you are showing us?

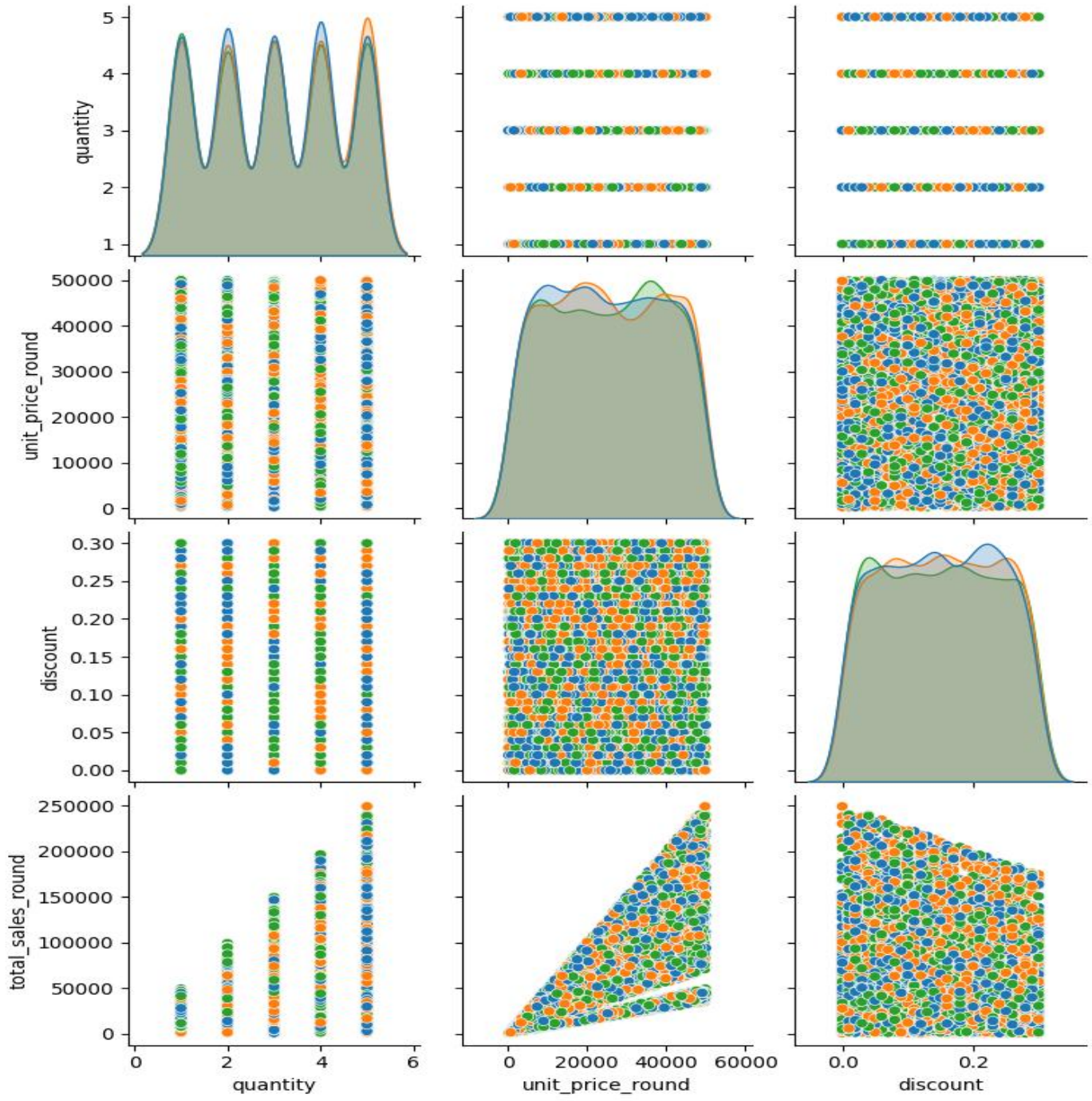
- No category is performing very low,which indicates that the business has a balanced performance across all categories.
- Electronics slightly leads, so it may generate more revenue on average
- Business can Focus more on Electronics for profit and improve Fashion & Home to match or exceed.

```
# Multivariate Analysis
# 6.Pairplot

sns.pairplot(df[['quantity', 'unit_price_round', 'discount',
'total_sales_round','category']],hue='category')

plt.show()
```

Output:



Interpretation of your above chart

Explain the Chart:

- The pairplot shows relationships between multiple variables:

1. Quanity

2. Unit Price
3. Discount
4. Total sales

- Each dot represents a transaction, and colors represent different categories (Home, Electronics, Fashion).
- Diagonal plots → show distribution (histogram)
- Off-diagonal plots → show relationships between variables (scatterplots)

What is it saying?

1. Relationship Insights:

- Unit Price vs Total Sales

> -> Strong positive relationship
>
> -> Higher price → higher total sales

- Quantity vs Total Sales -> Clear positive trend -> More quantity → more sales
- Discount vs Total Sales

> -> No strong pattern
>
> -> Discount has less direct impact on total sales

*Quantity vs Unit Price -> No clear relationship (independent variables)

2. Distribution Insights (Diagonal):

- Quantity - Discrete values (1 to 5)
- Unit Price - Almost uniform distribution
- Discount - Spread evenly between 0 to 0.3
- Total Sales- Right-skewed distribution (few high sales values)

3. Category Insights:

- Categories (Home, Electronics, Fashion) are evenly distributes,no major dominance of one category
- Slight variations of some categories appear more in higher sales ranges

What features you used?

- Pairplot → to analyze multiple variables at once
- Hue = 'category' to compare category wise behavior
- Scatterplots → to detect relationships

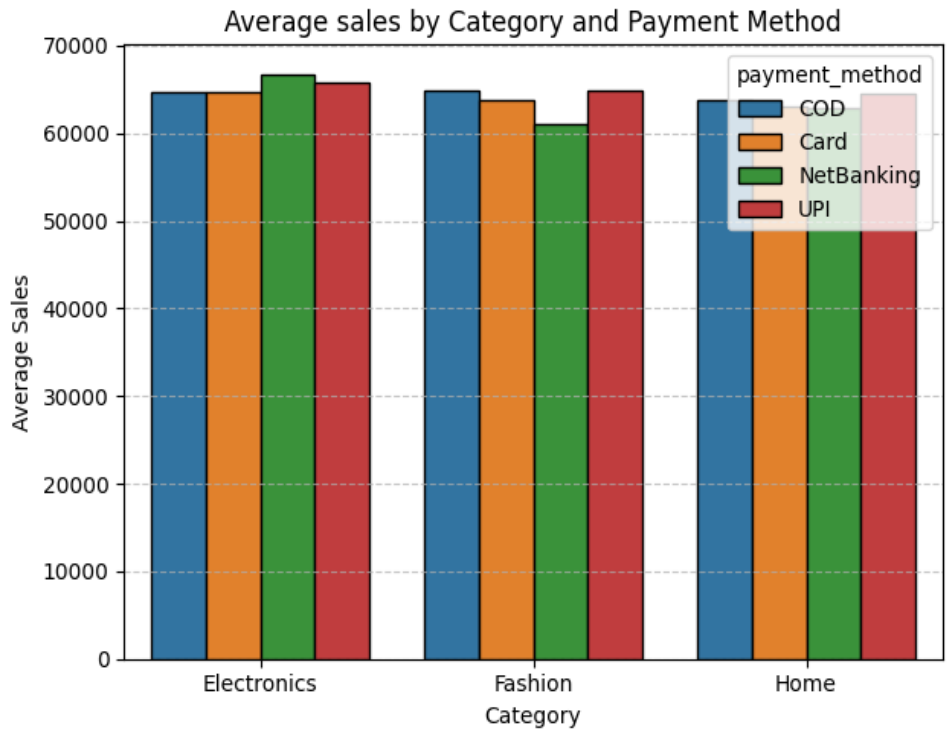
From this what you are showing us?

- Total Sales depends mainly on Unit Price,Quantity
- Discount does not strongly influence sales
- Categories perform similarly, indicating balanced product distribution
- The pairplot shows that total sales are strongly influenced by unit price and quantity, while discount has minimal impact. Categories perform similarly, and sales distribution is right-skewed.

#7.Grouped Analysis (Category + Payment Method)

```
grouped = df.groupby(['category',
'payment_method'])['total_sales_round'].mean().reset_index()
sns.barplot(x='category', y='total_sales_round', hue='payment_method',
data=grouped, edgecolor="black")
plt.title('Average sales by Category and Payment Method')
plt.xlabel('Category')
plt.ylabel('Average Sales')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Output:



Interpretation of your above chart

Explain the Chart:

- This bar chart shows the average total sales across different Categories (Electronics, Fashion, Home),Payment Methods (COD, Card, NetBanking, UPI)
- Each group of bars represents a category, and different colors represent payment methods.

What is it saying?

1. **Category-wise Insights:**

- Electronics

-> lightly higher average sales compared to other categories

-> NetBanking and UPI show better performance

- Fashion

-> Slightly lower average sales

-> NetBanking shows comparatively lower values

- Home

-> Moderate and consistent performance across all payment methods

2. **Payment Method Insights:**

- UPI and COD

-> Consistently show stable and good average sales

- Card

-> Slightly lower compared to others

- NetBanking

-> Performs well in Electronics but lower in Fashion

What features you used?

- Groupby() → to calculate average sales by category & payment method
- Barplot → to compare values across groups
- Hue (payment_method) → to differentiate payment types
- Edgecolor → for better bar visibility
- Grid (Y-axis) → to improve readability
- Tight layout → to avoid overlapping

From this what you are showing us?

- Sales performance is fairly consistent across payment methods
- Customers use all payment methods equally
- No strong dependency of sales on payment type
- Business can:

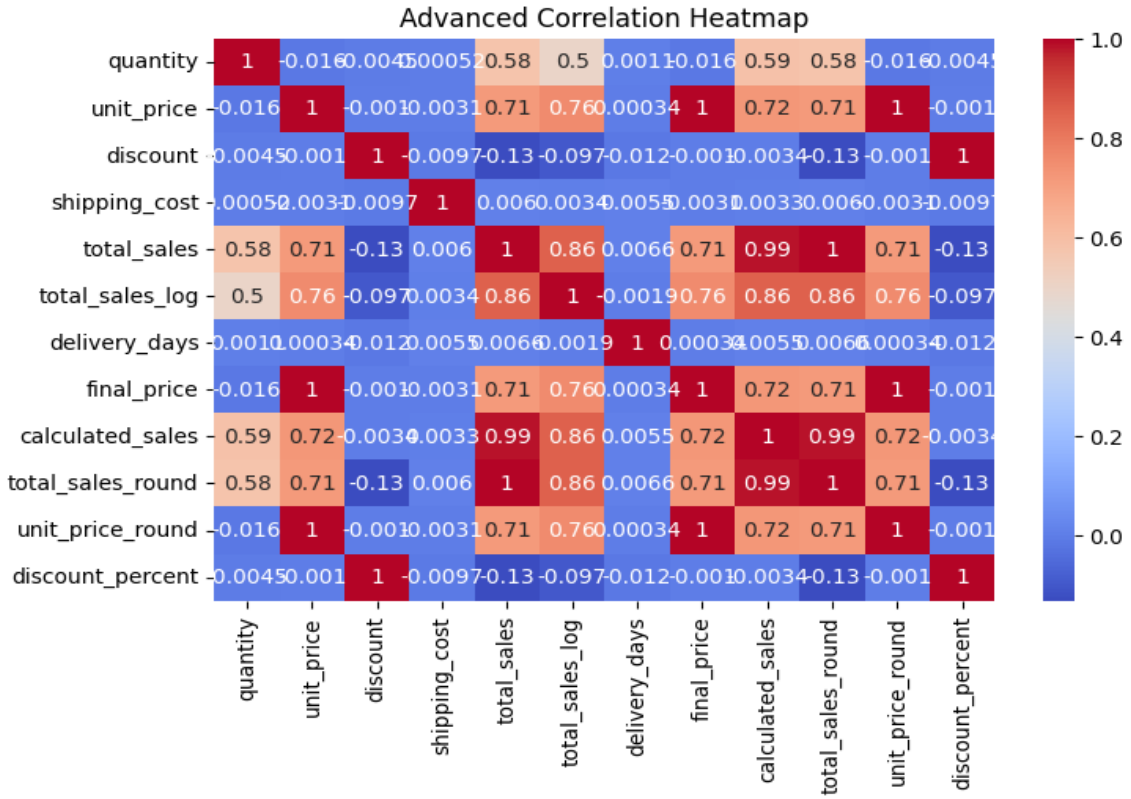
->Focus more on product/category strategy

->Ensure smooth experience across all payment options

8.Heatmap

```
plt.figure(figsize=(8,5))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Advanced Correlation Heatmap")
plt.show()
```

Output:



Interpretation of your above chart

Explain the Chart:

- This heatmap shows the correlation between numerical variables in the dataset.
- Values range from -1 to +1

+1 -> strong positive relationship
-1 -> strong negative relationship
0 -> no relationship

Color:

Red -> strong positive
Blue -> negative

What is it saying?

1. Strong Positive Correlations:

- Total Sales & Calculated Sales (~.0.99) - Almost identical ->derived feature
- Total Sales & Unit Price (~0.71) – Higher price->higher sales
- Total Sales & Quantity (~0.58) – More quanity -> more sales
- Unit Price & Final Price (~1.0) – Nearly perfect -> highly related variables

2. Moderate Correlations:

    - Total Sales & Total Sales Log (~0.86)
    - Calculated Sales & Unit Price (~0.72)

3. Negative Correlation:

- Discount & Total Sales (~ -0.13)
  -> Very weak negative relationship
  -> Discounts do not strongly increase sales

4. Weak / No Correlation:

- Shipping Cost, Delivery Days

    - Very low correlation with sales
    - Minimal impact on total sales

What features you used?

- df.corr() → to calculate correlation matrix
- numeric_only=True → only numerical columns
- Heatmap (Seaborn) → to visualize relationships
- annot=True -> to display correlation values
- cmap='coolwarm' -> color differentiation

From this what you are showing us?

- The heatmap shows that total sales are mainly influenced by unit price and quantity, while discount and operational factors have minimal impact. Some variables are highly correlated, indicating redundancy.

#9. Pivot table

```
pivot_table = pd.pivot_table(
    df,
    values='total_sales_round',
    index='discount',
    columns='category',
    aggfunc='mean',
    fill_value=0
)
pivot_table
```

Output:

category	Electronics	Fashion	Home
discount			
0.00	70429.928727	76461.992600	67671.172222
0.01	75953.115841	80052.038770	80274.306078
0.02	75776.786827	66268.974455	74641.893701
0.03	79393.876542	66002.243543	75931.540943
0.04	81809.838829	77412.469224	72515.113723
0.05	71756.906484	68378.255000	73166.621228
0.06	72411.755652	63027.444775	68311.346780
0.07	65336.404513	59090.601111	76917.636182
0.08	73551.116880	67877.179896	72231.291165
0.09	64492.232212	74168.672981	61258.749386
0.10	63059.369630	75465.274513	66299.384955
0.11	72346.580991	61829.262091	75369.190192
0.12	67865.594220	80086.241224	69638.399831
0.13	66736.882358	67626.617456	67305.992252
0.14	65167.067632	77630.485545	59521.512661
0.15	58593.405312	66049.934271	59182.916992
0.16	68239.373063	58328.917083	56754.132233
0.17	68860.034472	66361.491389	60318.548687
0.18	63185.729500	55793.016975	57065.989381
0.19	59628.960435	56538.772170	53388.324352
0.20	66895.899565	65613.126667	63701.099174
0.21	57742.597982	56650.946731	55433.781667
0.22	58087.075045	60155.761414	55926.045538
0.23	64975.903511	54579.236239	57801.407258
0.24	51887.240556	54970.654860	58040.914188
0.25	59611.661864	53471.905684	61930.544324
0.26	57526.799304	51239.501226	59330.398793
0.27	57142.148319	57030.810900	61146.211731
0.28	54040.562340	56760.599913	47112.698242
0.29	58158.016121	50342.119250	49221.576400
0.30	61964.539206	47194.072206	49009.615246

Interpretation of your above chart

Explain the Chart:

- This chart shows the average total sales for different discount levels across three product categories: Electronics, Fashion, and Home.
- It helps compare how sales vary when discount percentages change.

What is it saying?

- At lower discount levels (0.00-0.05),sales are generally higher
- As the discount increases, the average sales tend to decrease
- The trend shows a negative relationship between discount and sales
- Some fluctuations exist, but overall pattern is clear

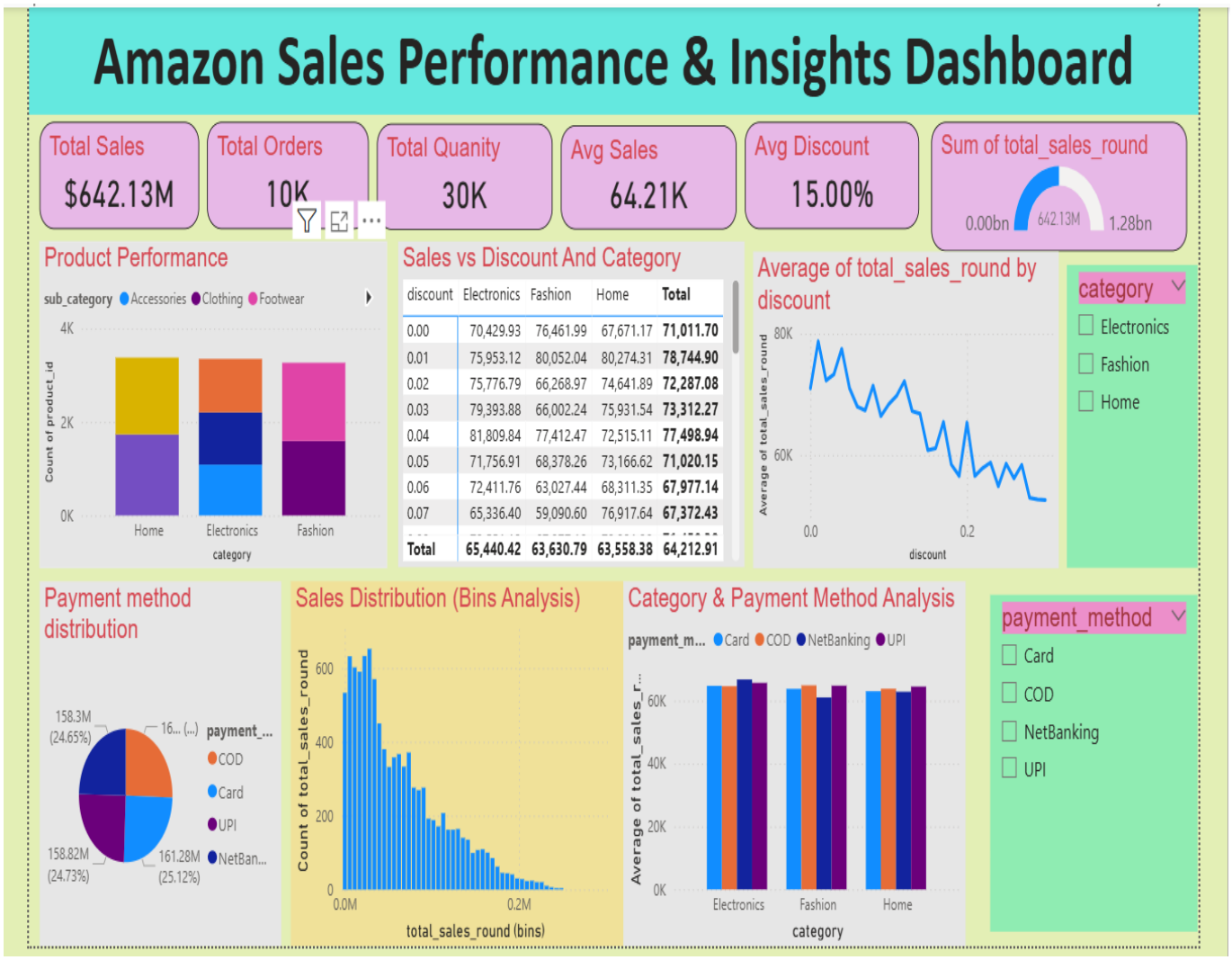
What features you used?

- discount → Used as index (rows)
- category → Used as columns (Electronics, Fashion, Home)
- total_sales_round → Used as values
- aggfunc = mean → Calculated average sales

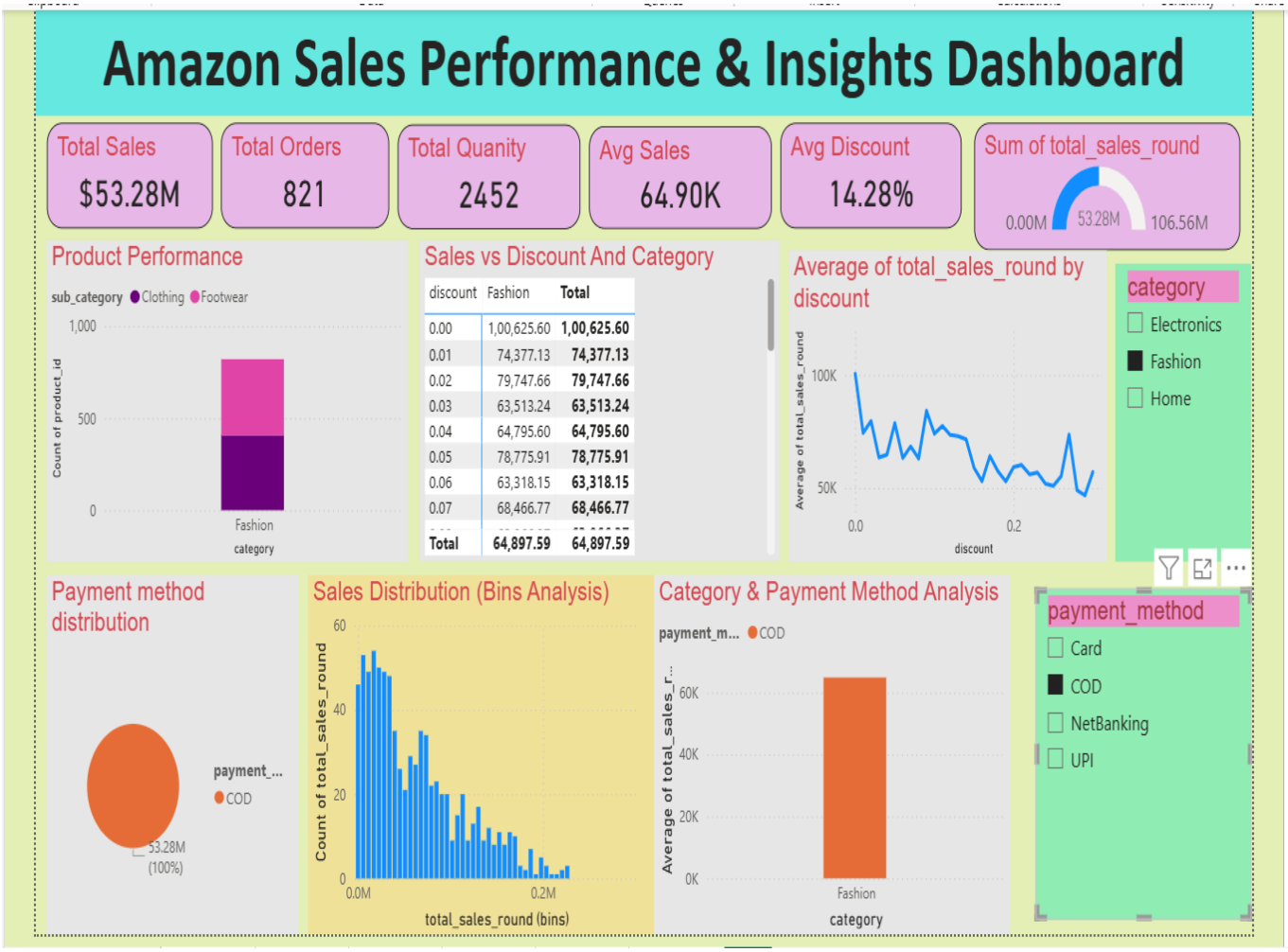
From this what you are showing us?

- Discounts do not always increase sales Higher discounts may reduce revenue instead of improving it
- Lower or moderate discounts are more effective
- Helps businesses decide optimal discount strategy

Dashboard


Amazon Sales Performance & Insights Dashboard
Total Sales
$642.13M
Total Orders
10K
Total Quanity
30K
Avg Sales
64.21K
Avg Discount
15.00%
Sum of total_sales_round
0.00bn
642.13M
1.28bn
Product Performance
sub_category Accessories Clothing Footwear
Count of product_id
4K
2K
0K
Home
Electronics
Fashion
category
Sales vs Discount And Category
discount Electronics Fashion Home Total
0.00 70,429.93 76,461.99 67,671.17 71,011.70
0.01 75,953.12 80,052.04 80,274.31 78,744.90
0.02 75,776.79 66,268.97 74,641.89 72,287.08
0.03 79,393.88 66,002.24 75,931.54 73,312.27
0.04 81,809.84 77,412.47 72,515.11 77,498.94
0.05 71,756.91 68,378.26 73,166.62 71,020.15
0.06 72,411.76 63,027.44 68,311.35 67,977.14
0.07 65,336.40 59,090.60 76,917.64 67,372.43
Total 65,440.42 63,630.79 63,558.38 64,212.91
Average of total_sales_round by discount
Average of total_sales_round
80K
60K
0.0
0.2
discount
category
Electronics
Fashion
Home
Payment method distribution
158.3M (24.65%)
16... (...)
payment_...
COD
Card
UPI
NetBan...
158.82M (24.73%)
161.28M (25.12%)
Sales Distribution (Bins Analysis)
Count of total_sales_round
600
400
200
0
0.0M
0.2M
total_sales_round (bins)
Category & Payment Method Analysis
payment_m... Card COD NetBanking UPI
Average of total_sales_r...
60K
40K
20K
0K
Electronics
Fashion
Home
category
payment_method
Card
COD
NetBanking
UPI

After Using Slicer in Dashboard



Insights From After Using Slicer in the Dashboard

1.Dashboard Overview

The Amazon Sales Performance & Insights Dashboard gives a full picture of:

- Sales performance
- Customer buying behavior
- Impact of discount
- Payment method usage

2.KPI Insights

The KPI section of the dashboard provides a quick summary of business performance, helping to understand revenue, order behavior, and pricing strategy.

1. Strong Revenue Performance

- Total Sales: $53.28M - The business is generating high overall revenue, indicating strong market performance and demand.

2. Low Order Volume but High Value

- Total Orders: 821
- Average Sales: 64.90K
- Even with a relatively lower number of orders, the average value per order is high.This means revenue is driven by high-value transactions rather than volume.

3. Moderate Product Movement

- Total Quantity: 2452
- The number of products sold is moderate compared to total sales.Indicates that products are priced higher, contributing to revenue.

4. Balanced Discount Strategy

- Average Discount: 14.28%
- The business is applying moderate discounts, not too high or too low.

5. Target Achievement Level

- Gauge Chart (Sales vs Target)
- Sales performance is around a mid-level target achievement.

.

3.Chart Insights

1. Product Performance (Stacked Column Chart)- Count of Product ID by Category & Sub-category

Insight:

- Only Fashion category visible with Clothing & Footwear
- High number of products in Fashion
- Product distribution is not balanced
- Business depends mainly on Fashion category

2.Sales vs Discount and Category (Matrix Table)

Insight:

- At 0% discount → highest sales (100K+)
- As discount increases → sales decrease
- Discounts are not improving sales
- Customers are buying even without discounts

3. Avg Sales by Discount (Line Chart)

Insight:

- Clear downward trend
- Higher discount → lower average sales
- Negative correlation between discount and sales
- Over-discounting may reduce profitability

4. Payment Method Distribution (Pie Chart)

Insight:

- 100% sales through COD

- Either:
    - Data issue (missing values), OR
    - Strong COD preference
- This is an anomaly

5. Sales Distribution Bins Analysis(Clusterd column Chart)

Insight:

- Most orders fall in low to mid sales range
- Very few high-value transactions
- Business relies on consistent mid-range orders
- High-value orders are rare but impactful

6. Category & Payment Method Analysis(Clusterd Chart)

Insight:

- Only Fashion + COD visible
- Avg sales around 65K
- No diversity in payment-category combination
- Again shows data limitation or imbalance

7. Slicers (Category & Payment Method)

Insight:

- Allows filtering by category and payment method
- Dashboard becomes interactive
- Helps in drill-down analysis

Future Enhancement

- The dashboard can be enhanced by integrating real-time data sources to enable live updates and faster decision-making.

- Adding profitability analysis will help evaluate actual business performance by including costs, discounts, and shipping charges.
- Customer segmentation can improve targeted marketing and customer retention strategies.
- Additionally, features like KPI alerts, mobile-friendly dashboards, and product-level analysis can further strengthen decision-making and overall business performance.

Conclusion

- This project successfully analyzes Amazon E-Commerce Sales & Customer Analytics by performing data cleaning, transformation, and exploratory data analysis (EDA) using Python, followed by the creation of an interactive Power BI dashboard.
- The dashboard provides clear insights into customer behavior, category performance, and sales trends, supporting data-driven decision-making.
- Overall, the project demonstrates how combining Python and Power BI can generate meaningful insights and improve business performance.